

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_98c1f280-b9f\agent-a80351dfba393eea6.jsonl`  
- SHA-256: `960550c9221a40e970e965db0b6f14fbede99a580c59e137f05f4cbb6e8e9ee1`  
- Source modified: `2026-06-14T06:33:55+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_98c1f280-b9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-14T06:32:15.324Z)

In the current badminton-highlight-indexer repo (cwd), audit ONE signal/component. Verify and return:

- present: the code exists.
- default_off: it does NOT run in the served/default path by default (e.g. cue_flags default False, default_segmenter is NOT 'fusion', the feature is not weighted by the shipped scorer unless explicitly enabled).
- no_special_casing: the signal enters the DECISION as a confidence-scored / feature COLUMN the learned scorer weights ? NOT a hard if/else branch hard-coded into the decision path.
- retained: NOTHING deletes or disables the code because the n=6 measured ?F1 was non-significant; the producer + feature math still exist and are re-measurable when the corpus grows.

Give evidence as concrete file:line refs with short quotes. List concerns. Be precise and skeptical; read the actual files.

SIGNAL: the AUDIO cue. Read backend/pipeline/detectors/audio_features.py (AUDIO_FEATURE_NAMES, compute_audio_features) and backend/pipeline/audio/hit_detector.py (detect_hits_in_video).

2. assistant (2026-06-14T06:32:18.450Z)

I'll audit the AUDIO cue signal. Let me start by reading the two core files specified, then trace how the audio cue flows into the decision path.

3. user (2026-06-14T06:32:19.452Z)

```
1      """Pure per-window audio-cue features for rally fusion (fusion P4).  
2  
3      NO I/O ? the audio hit detection lives in `backend/pipeline/audio/hit_detector.py`  
4      (`detect_hits_in_video` -> `List[HitEvent]`); this only turns those hit events into a  
few  
5      per-window features the learned scorer can weight. Mirrors `fusion_features.py`  
discipline.  
6  
7      Honest expectation (see the `audio-e1-findings` memory): the classical-onset audio cue  
had  
8      only ~2.2x discrimination and was NOT adopted standalone ? so this is "add the signal as  
a  
9      column and let the golden-set harness decide", default-OFF, not "adopt audio".  
10     ""  
11     from __future__ import annotations  
12  
13     from typing import Dict, List  
14  
15     AUDIO_FEATURE_NAMES = ["audio_hit_count", "audio_hit_rate", "audio_hit_strength_mean"]  
16  
17
```

```

18     def compute_audio_features(hits: List, window_start: float, window_end: float) ->
Dict[str, float]:
19         """Audio features for one window from a video's HitEvents (each has ``.t``,
``.strength``).
20
21         A real rally is a sustained volley of racket?shuttle contacts; dead time / a single
22         service feed has few. Returns hit count, hits/second, and mean strength ? all 0.0 on
an
23         empty window. ``[start, end)`` half-open so a hit on a boundary belongs to one
window only.
24         """
25         dur = max(1e-6, window_end - window_start)
26         in_win = [h for h in (hits or []) if window_start <= h.t < window_end]
27         if not in_win:
28             return {k: 0.0 for k in AUDIO_FEATURE_NAMES}
29         n = len(in_win)
30         return {
31             "audio_hit_count": float(n),
32             "audio_hit_rate": n / dur,
33             "audio_hit_strength_mean": sum(h.strength for h in in_win) / n,
34         }
35

```

4. user (2026-06-14T06:32:19.678Z)

```

1     """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).
2
3     Stages 0?1 of docs/AUDIO_RALLY_DETECTION_PLAN.md: extract the audio track, build a
4     spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient
5     a racket?shuttle contact makes), threshold it adaptively, and return (timestamp,
6     strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
7     only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion
8     (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.
9     """
10    from __future__ import annotations
11
12    import os
13    import wave
14    import tempfile
15    import subprocess
16    from dataclasses import dataclass
17    from typing import List, Optional, Tuple
18
19    import numpy as np
20
21
22    @dataclass
23    class HitEvent:
24        t: float          # seconds
25        strength: float    # onset-envelope peak value (arbitrary units)
26
27
28    def extract_audio(video_path: str, out_wav: Optional[str] = None, sr: int = 16000) ->
str:
29        """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
30        Audio-only decode, so it's fast even for multi-GB video."""
31        if out_wav is None:
32            fd, out_wav = tempfile.mkstemp(suffix=".wav")
33            os.close(fd)
34            cmd = ["ffmpeg", "-y", "-i", video_path, "-vn", "-ac", "1", "-ar", str(sr),
35                  "-c:a", "pcm_s16le", out_wav]
36            subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
37            return out_wav
38

```

```

39
40 def load_wav(path: str) -> Tuple[np.ndarray, int]:
41     """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
42     with wave.open(path, "rb") as w:
43         sr, n, sw, ch = w.getframerate(), w.getnframes(), w.getsampwidth(),
w.getnchannels()
44         raw = w.readframes(n)
45         if sw == 2:
46             a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
47         elif sw == 4:
48             a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
49         elif sw == 1:
50             a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
51         else:
52             raise ValueError(f"unsupported sample width {sw}")
53         if ch > 1:
54             a = a.reshape(-1, ch).mean(axis=1)
55         return a, sr
56
57
58 def _preemphasis(x: np.ndarray, coef: float = 0.97) -> np.ndarray:
59     if len(x) < 2:
60         return x.copy()
61     return np.append(x[0], x[1:] - coef * x[:-1])
62
63
64 def onset_envelope(samples: np.ndarray, sr: int, frame: int = 1024,
65                    hop: int = 256, band: Optional[Tuple[float, float]] = None) ->
Tuple[np.ndarray, float]:
66     """Spectral-flux onset strength per hop (pure-numpy STFT). Returns (env,
hop_seconds).
67
68     `band=(lo_hz, hi_hz)` restricts the flux to a frequency band ? a shuttle "thwack"
69     is HF-rich while voices/footsteps live lower, so band-limiting (e.g. 3-9 kHz)
70     suppresses non-shuttle transients. Needs sr high enough for hi_hz (Nyquist =
sr/2)."""
71     x = _preemphasis(np.asarray(samples, dtype=np.float32))
72     hop_sec = hop / float(sr)
73     if len(x) < frame:
74         return np.zeros(0, dtype=np.float32), hop_sec
75     win = np.hanning(frame).astype(np.float32)
76     n_frames = 1 + (len(x) - frame) // hop
77     idx = np.arange(frame)[None, :] + hop * np.arange(n_frames)[: , None]
78     frames = x[idx] * win
79     mag = np.abs(np.fft.rfft(frames, axis=1))
80     if band is not None:
81         freqs = np.fft.rfftfreq(frame, d=1.0 / sr)
82         keep = (freqs >= band[0]) & (freqs <= band[1])
83         if keep.any():
84             mag = mag[:, keep] # band-limited flux
85     diff = np.diff(mag, axis=0) # frame-to-frame change
86     flux = np.sqrt((np.maximum(diff, 0.0) ** 2).sum(axis=1)) # positive (onset) flux
87     env = np.concatenate([[0.0], flux]).astype(np.float32) # align to n_frames
88     return env, hop_sec
89
90
91 def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
92     """Sliding-window mean and std (reflect-padded, via cumsum). win forced odd."""
93     if win % 2 == 0:
94         win += 1
95     pad = win // 2
96     p = np.pad(env.astype(np.float64), pad, mode="reflect")
97     c1 = np.cumsum(np.insert(p, 0, 0.0))
98     c2 = np.cumsum(np.insert(p * p, 0, 0.0))
99     s1 = c1[win:] - c1[:-win]

```

```

100     s2 = c2[win:] - c2[:-win]
101     mean = s1 / win
102     var = np.maximum(s2 / win - mean * mean, 0.0)
103     return mean[:len(env)], np.sqrt(var)[:len(env)]
104
105
106     def detect_hits(samples: np.ndarray, sr: int, k: float = 2.5, win_frames: int = 41,
107                    min_gap_sec: float = 0.08, band: Optional[Tuple[float, float]] = None)
-> List[HitEvent]:
108         """Detect impact transients: local maxima of the onset envelope that exceed an
109         adaptive threshold (moving mean + k*std), keeping the strongest within
110         `min_gap_sec`.
111         `k` is the sensitivity knob (lower = more hits); `band` band-limits the onset flux
112         to the shuttle-click frequencies (e.g. (3000, 9000))."""
113         env, hop_sec = onset_envelope(samples, sr, band=band)
114         if len(env) < 3:
115             return []
116         mean, std = _moving_stats(env, win_frames)
117         thr = mean + k * std
118         is_peak = (env[1:-1] >= env[:-2]) & (env[1:-1] > env[2:]) & (env[1:-1] > thr[1:-1])
119         peak_idx = np.nonzero(is_peak)[0] + 1
120         if peak_idx.size == 0:
121             return []
122         # enforce min inter-peak gap: greedily keep strongest first
123         min_gap = max(1, int(round(min_gap_sec / hop_sec)))
124         order = peak_idx[np.argsort(-env[peak_idx])]
125         taken = np.zeros(len(env), dtype=bool)
126         kept = []
127         for i in order:
128             lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
129             if taken[lo:hi].any():
130                 continue
131             taken[i] = True
132             kept.append(i)
133         kept.sort()
134         return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]
135
136     def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
137         """Convenience: extract audio from a video and detect hits. Cleans up the temp
138         wav."""
139         wav = extract_audio(video_path, sr=sr)
140         try:
141             samples, real_sr = load_wav(wav)
142             return detect_hits(samples, real_sr, **kw)
143         finally:
144             try:
145                 os.remove(wav)
146             except OSError:
147                 pass

```

5. assistant (2026-06-14T06:32:23.668Z)

Both core files read. Now let me trace how the audio cue enters the decision path ? the scorer, the fusion segmenter, the cue flags/config, and check whether the default segmenter is fusion. Let me search for the consumers.

6. user (2026-06-14T06:32:25.406Z)

```

backend\pipeline\detectors\audio_features.py:4:(`detect_hits_in_video` -> `List[HitEvent]`);
this only turns those hit events into a few
backend\pipeline\detectors\audio_features.py:15:AUDIO_FEATURE_NAMES = ["audio_hit_count",
"audio_hit_rate", "audio_hit_strength_mean"]
backend\pipeline\detectors\audio_features.py:18:def compute_audio_features(hits: List,

```

```

window_start: float, window_end: float) -> Dict[str, float]:
backend\pipeline\detectors\audio_features.py:28:         return {k: 0.0 for k in
AUDIO_FEATURE_NAMES}
backend\pipeline\detectors\audio_features.py:31:         "audio_hit_count": float(n),
backend\pipeline\detectors\audio_features.py:32:         "audio_hit_rate": n / dur,
backend\pipeline\detectors\audio_features.py:33:         "audio_hit_strength_mean":
sum(h.strength for h in in_win) / n,
backend\pipeline\audio\hit_detector.py:136:def detect_hits_in_video(video_path: str, sr: int =
16000, **kw) -> List[HitEvent]:
backend\eval\fusion_audio.py:4:`detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and
adds the 3 audio features per
backend\eval\fusion_audio.py:25:from backend.pipeline.audio.hit_detector import
detect_hits_in_video
backend\eval\fusion_audio.py:40:         hits = detect_hits_in_video(video, k=k)
backend\eval\fusion_audio.py:44:         c["audio"] = af.compute_audio_features(hits,
c["start"], c["end"])
backend\eval\fusion_audio.py:68:         for fn in af.AUDIO_FEATURE_NAMES:
backend\eval\fusion_audio.py:84:         feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES) + list(af.AUDIO_FEATURE_NAMES),
backend\eval\audio\e1_probe.py:18:from backend.pipeline.audio.hit_detector import
detect_hits_in_video, HitEvent
backend\eval\audio\e1_probe.py:55:     hits = detect_hits_in_video(video, k=k)
docs\CODE_MAP.md:153:[Omitted long matching line]
tests\test_audio_features.py:16:def test_compute_audio_features_empty():
tests\test_audio_features.py:17:     f = af.compute_audio_features([], 0.0, 10.0)
tests\test_audio_features.py:18:     assert set(f) == set(af.AUDIO_FEATURE_NAMES)
tests\test_audio_features.py:22:def test_compute_audio_features_counts_rate_strength():
tests\test_audio_features.py:24:     f = af.compute_audio_features(hits, 0.0, 10.0)
tests\test_audio_features.py:25:     assert f["audio_hit_count"] == 3.0
tests\test_audio_features.py:26:     assert f["audio_hit_rate"] == pytest.approx(0.3)
# 3 hits / 10 s
tests\test_audio_features.py:27:     assert f["audio_hit_strength_mean"] == pytest.approx((2 + 4
+ 6) / 3)
tests\test_audio_features.py:30:def test_compute_audio_features_half_open_boundary():
tests\test_audio_features.py:32:     f = af.compute_audio_features(hits, 10.0, 20.0)
tests\test_audio_features.py:33:     assert f["audio_hit_count"] == 1.0
tests\test_audio_features.py:39:     """shuttle + person constant (no signal); audio_hit_count
separates rallies (8) from junk (0)."""
tests\test_audio_features.py:44:         audio = {fn: 0.0 for fn in af.AUDIO_FEATURE_NAMES}
tests\test_audio_features.py:45:         audio["audio_hit_count"] = 8.0 if pos else 0.0

```

7. user (2026-06-14T06:32:26.231Z)

```

config.json:26:         "default_segmenter": "gemini",
docs\ANALYZERS_AND_RECONCILERS.md:204:Flag-gated, **default OFF** via the existing
`indexing.fusion.cue_flags` (a sibling `analyzer_flags`
docs\ANALYZERS_AND_RECONCILERS.md:425:- Everything inherits fusion P2's control-reproducing
defaults, so the fusion segmenter with all flags off
docs\CODE_MAP.md:31: -> seg_name = req.segmenter or global_config.indexing.default_segmenter
(fallback 'motion')
docs\CODE_MAP.md:89: - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
docs\CODE_MAP.md:99:[Omitted long matching line]
docs\CODE_MAP.md:105:- `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
docs\CODE_MAP.md:113:[Omitted long matching line]
docs\CODE_MAP.md:114:[Omitted long matching line]
docs\CODE_MAP.md:129:[Omitted long matching line]
docs\CODE_MAP.md:131:[Omitted long matching line]
docs\CODE_MAP.md:186:[Omitted long matching line]
docs\CODE_MAP.md:250:- **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run*.py read the model value (no literals); `main.py` keeps the defensive `motion`

```

```

getattr fallback noted above.
docs\CODE_MAP.md:276:| Change a config default / add a config field |
`@backend/config/models.py` (single source of truth) -> regenerate via
`setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
backend\config\models.py:81:    Every default reproduces control behaviour: the fusion segmenter
persists the
backend\config\models.py:112:    default_segmenter: str = "yolo_hybrid"
backend\config\models.py:142:    fusion: FusionConfig = Field(default_factory=FusionConfig)
backend\config\models.py:144:    @field_validator("default_segmenter")
docs\EXPERIMENT_HARNESS.md:106:- `backend/config/models.py::IndexingConfig` ?
`default_segmenter` (:65, default `yolo_hybrid`), `detector_model` (:72),
`detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75),
`min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). **No
`min_crossings` knob yet** (see fusion P2).
docs\EXPERIMENT_HARNESS.md:123:JSON-serializable; selected via the existing `segmenter_registry`
+ `IndexingConfig.default_segmenter`.
docs\EXPERIMENT_HARNESS.md:155:pinned ? **rollback = flip `default_segmenter`/variant config,
never a code revert.**
docs\EXPERIMENT_HARNESS.md:163:- **Control variant pinned** ? frozen recipe + classifier model
hash, always registered & default.
docs\EXPERIMENT_HARNESS.md:185:3. **Per-cue enable flags** in `IndexingConfig` (default
`False`); `default_segmenter` already exists.
README.md:74:This outputs a validation summary and indexes all rallies directly into the local
SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json`. **The
shipped `config.json` sets `gemini`** (needs `GEMINI_API_KEY`; without it the run silently
produces 0 segments and the observability report flags `silent_provider_noop`). Set it to
`yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.
README.md:142:    "default_segmenter": "gemini",
README.md:180:* `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
README.md:242:    *Or set `default_segmenter`: "yolo_hybrid" (or `gemini`/`motion`) under
`indexing` in `config.json`.*
README.md:246:[Omitted long matching line]
README.md:264:    "default_segmenter": "gemini",
run_process_and_stitch.py:74:    seg_name = cfg.indexing.default_segmenter
docs\MULTI_SIGNAL_FUSION_PLAN.md:28:| `yolo_hybrid` (the current `default_segmenter`) | person
velocity (torchvision + ByteTrack) | `backend/pipeline/segmenters/yolo_hybrid.py` |
docs\MULTI_SIGNAL_FUSION_PLAN.md:271:    **default-OFF** ? `default_segmenter` unchanged),
subclassing the trajectory engine via two identity
docs\MULTI_SIGNAL_FUSION_PLAN.md:276:    pure + unit-tested). Served gating is optional + default-
OFF: `fusion.min_crossings` (Gate A) /
backend\reporting\spec.yaml:133:    The segmenter that ran differs from config.json's
default_segmenter, and no
backend\reporting\harvest.py:216:    config_default = (config.get("indexing") or
{}).get("default_segmenter")
docs\QUALITY_ITERATIONS.md:68:candidate set; flipping the **live served default** stays owner-
gated until a real fusion-path run confirms it
docs\QUALITY_ITERATIONS.md:139:**Shipping shape.** A NEW `fusion` segmenter, **default OFF**
(`default_segmenter` unchanged); Gate A
docs\QUALITY_ITERATIONS.md:146:**Status (2026-06-14): SHIPPED, default-OFF.** `FusionSegmenter`
(`backend/pipeline/segmenters/fusion_hybrid.py`,
docs\QUALITY_ITERATIONS.md:152:the extended equivalence test); default `FusionSegmenter` == its
wasb substrate. The confirmed low/medium findings
docs\TRACKNET_WSL_SETUP.md:71:Then select it as the segmenter (e.g. set
`indexing.default_segmenter` to
tests\test_config.py:23:    assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:78:        "default_segmenter": "gemini",
tests\test_config.py:89:    assert cfg.indexing.default_segmenter == "gemini"
tests\test_config.py:102:        "indexing": {"default_segmenter": "yolo_hybrid"},
tests\test_config.py:110:    assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:138:        "indexing": {"default_segmenter": "gemini", "wasb":
{"velocity_threshold": 0.02}},
tests\test_config.py:154:    assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:198:    assert cfg.get("indexing", {}).get("default_segmenter") ==
"yolo_hybrid"
backend\main.py:148:    default_seg = getattr(getattr(global_config, "indexing", None),

```

```

"default_segmenter", "motion")
backend/main.py:712:         seg_name = args.segmenter or getattr(getattr(global_config,
"indexing", None), "default_segmenter", "motion")
backend/pipeline/segmenters/fusion_hybrid.py:24:and hands off identically to `wasb_hybrid`.
Registered as ``fusion_hybrid``, NOT the default
backend/pipeline/validators/_init__.py:9:# Register all default validators in sequence
(dependencies first)
tests/test_reporting_harvest.py:46:         config={"ai_provider": "gemini", "ai_model": "gemini-
flash", "indexing": {"default_segmenter": "wasb_hybrid"}},
tests/test_reporting_harvest.py:122:         config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "gemini"}},
tests/test_reporting_harvest.py:137:         config={"indexing": {"default_segmenter":
"motion"}})
tests/test_reporting_harvest.py:146:     cfg = {"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}}
tests/test_reporting_harvest.py:189:         config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}},

```

8. assistant (2026-06-14T06:32:31.212Z)

Key finding: the audio cue consumers are in `backend/eval/fusion\_audio.py` and `backend/eval/audio/e1\_probe.py` ? both eval-only. The production segmenter is `fusion\_hybrid.py`, which is NOT the default. Let me check whether `fusion\_hybrid.py` (the served fusion segmenter) consumes audio features at all, and inspect `fusion\_audio.py` and the config cue_flags.

9. user (2026-06-14T06:32:32.177Z)

```

1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13     torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14     model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15     via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16     - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
experiment
20     harness can re-score any variant.
21
22     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """

```

```

28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
41 B)."""
42
43         family = "fusion"
44         display_label = "Fusion"
45
46         def __init__(self, db: Any, config: Any):
47             super().__init__(db, config)
48             # Built lazily only if the person cue is enabled (no detector model loaded
49 otherwise).
50             self._person: Optional[Any] = None
51             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
52 estimate it
53             # over the whole trajectory once per candidate window (it depends only on
54 `points`).
55             self._axis_cache_key: Optional[int] = None
56             self._axis_cache: Optional[tuple] = None
57
58         @property
59         def name(self) -> str:
60             return "fusion_hybrid"
61
62         @property
63         def description(self) -> str:
64             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
65 rallies, "
66
67 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
68 and the "
69
70 "AI provider sets semantic boundaries.")
71
72         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
73 Any]]:
74             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
75             if substrate == "tracknet":
76                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
77                 return TrackNetRunner(cfg), {
78                     "weights_path": cfg.weights_path,
79                     "queue_length": cfg.queue_length,
80                     "fusion_substrate": "tracknet",
81                 }
82             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
83             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
84 "wasb"}
85
86         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
87 frame_width: float, video_path: str,
88 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
89             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
90 video_path, idx_cfg)
91             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
92 persisted
93             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
94 estimated

```



```

82 # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83 gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84 min_crossings=0,
estimate=self._axis_estimate(points))
85 stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87 fcfg = idx_cfg.get("fusion", {})
88 if fcfg.get("cue_flags", {}).get("person", False):
89     stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90     return stats
91
92 def _axis_estimate(self, points: List[Any]) -> tuple:
93     """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94     computed once per video, not once per candidate window (it depends only on
points)."""
95     key = id(points)
96     if self._axis_cache_key != key or self._axis_cache is None:
97         self._axis_cache_key = key
98         self._axis_cache = rally_gate.estimate_play_axis(points)
99     est = self._axis_cache
100     assert est is not None
101     return est
102
103 def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104 frame_width: float, video_path: str,
105 fcfg: Dict[str, Any]) -> Dict[str, float]:
106     """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107     frame range and compute the scale/translation-invariant person features. Lazily
builds
108     ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109     # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110     from backend.pipeline.detectors.person_detector import PersonDetector
111     if self._person is None:
112         self._person = PersonDetector(self.config)
113
114     frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115     # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116     # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117     start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118     det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119     fw = det["frame_width"] or frame_width
120     fh = det["frame_height"]
121     # If we can't trust the frame dims, every normalised feature would silently
collapse
122     # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123     if fw <= 0 or fh <= 0:
124         return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126     poly = fcfg.get("court_polygon")
127     polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128     net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129     shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130     return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                     court_polygon=polygon, net=net)
132
133 def _select_for_handoff(self, windows: List[Dict[str, Any]],
134 window_stats: List[Dict[str, Any]]):

```

```

135             idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are
138         untouched (full superset)."""
139         fcfg = idx_cfg.get("fusion", {})
140         min_crossings = int(fcfg.get("min_crossings", 0))
141         min_players = int(fcfg.get("min_players", 0))
142         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150             keep.append(i)
151         return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155

```

10. user (2026-06-14T06:32:32.640Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4     `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5     window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6     and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7     of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8     (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10    Run (no GPU):
11        python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12    """
13    from __future__ import annotations
14
15    import argparse
16    import glob
17    import json
18    import os
19    from typing import Any, Dict, List
20
21    from backend.eval import experiment
22    from backend.eval.fusion_compare import delta_stats
23    from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
_derive_video_path
24    from backend.pipeline.detectors import audio_features as af
25    from backend.pipeline.audio.hit_detector import detect_hits_in_video
26
27
28    def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
29        """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
30        with open(manifest_path, encoding="utf-8") as f:

```

```

31         specs = json.load(f)
32     written: List[str] = []
33     for spec in specs:
34         jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
35         if not os.path.isfile(jp):
36             print(f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping")
37             continue
38         video = _derive_video_path(spec)
39         print(f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
flush=True)
40         hits = detect_hits_in_video(video, k=k)
41         with open(jp, encoding="utf-8") as f:
42             data = json.load(f)
43             for c in data["candidates"]:
44                 c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
45             with open(jp, "w", encoding="utf-8") as f:
46                 json.dump(data, f, indent=2)
47             print(f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows", flush=True)
48             written.append(jp)
49     return written
50
51
52 def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
53     """Load the feature JSONs into VideoCandidates including the audio columns (requires
54     `augment` to have run ? raises if a JSON lacks audio)."""
55     videos: List[experiment.VideoCandidates] = []
56     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
57         with open(path, encoding="utf-8") as f:
58             d = json.load(f)
59             cands = d["candidates"]
60             if cands and "audio" not in cands[0]:
61                 raise SystemExit(f"{path} has no audio features ? run `--augment` first")
62             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
63             features: Dict[str, List[float]] = {}
64             for fn in SHUTTLE_FEATURE_NAMES:
65                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
66             for fn in PERSON_FEATURE_NAMES:
67                 features[fn] = [float(c["person"][fn]) for c in cands]
68             for fn in af.AUDIO_FEATURE_NAMES:
69                 features[fn] = [float(c["audio"][fn]) for c in cands]
70             gts = [(float(a), float(b)) for a, b in d["gts"]]
71             videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
72                                                         features=features, gts=gts))
73     return videos
74
75
76 def compare_increment(videos: List[experiment.VideoCandidates], iou: float = 0.5,
77                       threshold: float = 0.5) -> Dict[str, Any]:
78     """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
79     LOV0)."""
80     person = experiment.Variant(
81         name="shuttle_person",
82         feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
83         threshold=threshold)
84     person_audio = experiment.Variant(
85         name="shuttle_person_audio",
86         feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES) +
87         list(af.AUDIO_FEATURE_NAMES),
88         threshold=threshold)
89     return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
90
91
92 def format_result(res: Dict[str, Any], iou: float) -> str:

```

```

90     a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
91     ds = delta_stats(res)
92     st = ds["sign_test"]
93     lines = [
94         f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou},
95 honest) ===",
96         f"  shuttle+person          : F1={a['f1']:.3f}  P={a['precision']:.3f}
R={a['recall']:.3f}",
97         f"  shuttle+person+audio : F1={b['f1']:.3f}  P={b['precision']:.3f}
R={b['recall']:.3f}",
98         f"  incremental ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)",
99         f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
100        f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
101        f"| CI clears 0: {ds['ci_excludes_zero']}",
102    ]
103    for pv in res["per_video_delta"]:
104        lines.append(f"      {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
105    return "\n".join(lines)
106
107    def main() -> None:
108        ap = argparse.ArgumentParser(description="Fusion P4 ? incremental audio lift on the
golden set.")
109        ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
110        ap.add_argument("--features-dir", default="output")
111        ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
112        ap.add_argument("--augment", action="store_true", help="add audio features to the
existing JSONs")
113        ap.add_argument("--compare", action="store_true", help="report the incremental audio
LOVO ?F1")
114        ap.add_argument("--iou", type=float, default=0.5)
115        ap.add_argument("--record", action="store_true")
116        args = ap.parse_args()
117
118        if args.augment:
119            augment(args.manifest, args.features_dir, args.k)
120        if args.compare:
121            videos = load_videos_with_audio(args.features_dir)
122            if len(videos) < 2:
123                raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
124            res = compare_increment(videos, iou=args.iou)
125            print(format_result(res, args.iou))
126            if args.record:
127                row = experiment.record_experiment(res)
128                print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
129
130
131    if __name__ == "__main__":
132        main()
133

```

11. assistant (2026-06-14T06:32:37.384Z)

Now I have a clear picture. The critical finding: the production `FusionSegmenter` (`fusion_hybrid.py`) does NOT consume audio features at all ? `\_enrich\_candidate\_stats` adds `net\_crossings` and (optionally) person features, but never calls `compute\_audio\_features` or `detect\_hits\_in\_video`. The audio cue lives ENTIRELY in the eval harness (`fusion_audio.py`). Let me verify the config side (FusionConfig, cue_flags ? is there an audio flag?) and confirm the learned scorer treats audio as a feature column.

12. user (2026-06-14T06:32:38.809Z)

```

61     wsl_distro: str = "Ubuntu"
62     repo_dir: str = "~/models/WASB-SBDT"
63     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64     conda_env: str = "wasb"
65     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66     sport: str = "badminton"
67     wsl_stage_dir: str = "~/clips"
68     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69     velocity_threshold: float = 0.02
70     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73     keep_frames: bool = False
74     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75     min_free_gb: float = 0.0
76
77
78     class FusionConfig(BaseModel):
79         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81         Every default reproduces control behaviour: the fusion segmenter persists the
82         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
83         true ? the person-cue features, but it does NOT gate the served handoff unless a
84         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
85         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
86         supplied ? off-court persons (spectators / adjacent courts) are excluded.
87         """
88         # Which trajectory substrate seeds the windows (shuttle stays primary).
89         substrate: Literal["wasb", "tracknet"] = "wasb"
90         # Windowing velocity threshold for the fusion family (the engine reads
91         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
92         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
93         velocity_threshold: float = 0.02
94         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
95         # enabled ? so the default path never imports torch / touches the GPU.
96         cue_flags: dict[str, bool] = Field(default_factory=dict)
97         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
98         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
99         min_crossings: int = Field(default=0, ge=0)
100         # Served Gate B: when the person cue is on, drop windows with median in-court
players
101         # < this. 0 = OFF.
102         min_players: int = Field(default=0, ge=0)
103         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104         court_polygon: Optional[list[list[float]]] = None
105         # Net line for the person two-sided-motion split (person features only ? the shuttle
106         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
107         net_axis: Literal["x", "y"] = "y"
108         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
109
110
111     class IndexingConfig(BaseModel):
112         default_segmenter: str = "yolo_hybrid"
113         use_gpu: bool = True
114         motion_threshold: float = 0.08

```

```

115     min_segment_duration: float = 3.0
116     dead_time_merge_gap: float = 2.0
117     chunk_duration: float = 90.0
118     chunk_overlap: float = 10.0
119     detector_model: str = "fasterrcnn_resnet50_fpn"
120     detector_score_threshold: float = 0.5
121     yolo_velocity_threshold: float = 0.15
122     yolo_frame_skip: int = 3
123     yolo_tracker: str = "bytetrack.yaml"
124     yolo_prefetch_workers: int = 2
125     min_action_window_duration: float = 2.0
126     log_yolo_candidates: bool = True
127     store_yolo_candidates: bool = True
128     ai_handoff_padding: float = 2.0
129     ai_max_workers: int = 5
130     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
131     # chunks of a high-bitrate (4K) source blow past the provider upload cap
132     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
133     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
134     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
135     # Matches gemini_refine's --clip-scale-width default.
136     ai_chunk_scale_width: int = 1280
137     # Optional debug knob: trim every video to the first N seconds before processing.
138     debug_duration: Optional[float] = None
139
140     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
141     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
142     fusion: FusionConfig = Field(default_factory=FusionConfig)
143
144     @field_validator("default_segmenter")
145     @classmethod
146     def segmenter_must_be_registered(cls, v: str) -> str:
147         # Registry check happens at runtime in main/segmenter selection.
148         # Here we just allow any string for forward compatibility.
149         return v

```

13. user (2026-06-14T06:32:39.627Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A Variant is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.

```

```

28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
ratio
56     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
scores
57
58     logger = logging.getLogger(__name__)
59
60     # Where a comparison run appends one reproducible record. Tracked location (NOT under
61     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62     # regression.DEFAULT_BASELINE_PATH.
63     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64     _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67     class ExperimentError(ValueError):
68         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69         score an unlabeled video with no pinned model, or a gate referencing an absent
70         feature)."""
71
72
73     # ----- Variant
74
75
76     @dataclass
77     class Variant:
78         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80         A variant turns one video's persisted candidate windows + features into rally
81         intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82         filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83         to control ? the core no-regression guarantee.
84
85         Fields:
86         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87           for the leaderboard and for selecting the served path (the existing
88           `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89           New cues are recorded here default-OFF.
90         - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted

```

```

91         ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92         ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93         windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for ``compare_unlabeled`` on a learned variant.
100    - ``feature_names`` ? the persisted features the learned filter consumes. When set
101      WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102      (honest) ? the recipe, not a pinned artifact.
103    - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104      overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105    """
106
107    name: str
108    segmenter: str = "wasb_hybrid"
109    config_overrides: Dict[str, Any] = field(default_factory=dict)
110    cue_flags: Dict[str, bool] = field(default_factory=dict)
111    min_crossings: int = 0
112    min_players: int = 0
113    classifier_model: Optional[str] = None
114    feature_names: Optional[List[str]] = None
115    threshold: float = 0.5
116    merge_gap: float = 1.0
117    # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118    # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119    # failure where a fixed 0.5 zeroed out a small-candidate video.
120    tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121    calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123    def is_learned(self) -> bool:
124        """True if this variant applies a learned classifier (pinned model or
recipe)."""
125        return self.classifier_model is not None or bool(self.feature_names)
126
127    def to_dict(self) -> dict:
128        return {
129            "name": self.name,
130            "segmenter": self.segmenter,
131            "config_overrides": self.config_overrides,
132            "cue_flags": self.cue_flags,
133            "min_crossings": self.min_crossings,
134            "min_players": self.min_players,
135            "classifier_model": self.classifier_model,
136            "feature_names": self.feature_names,
137            "threshold": self.threshold,
138            "merge_gap": self.merge_gap,
139            "tune_threshold": self.tune_threshold,
140            "calibrate": self.calibrate,
141        }
142
143    @classmethod
144    def from_dict(cls, d: dict) -> "Variant":
145        known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146        return cls(**{k: v for k, v in d.items() if k in known})
147
148    def spec_hash(self) -> str:
149        """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150        and makes a result reproducible. The pinned **control** variant always hashes

```



```

the
151         same, so a regression is always traceable to a recipe change, never to drift."""
152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160
161     # ----- persisted candidates
162
163
164     @dataclass
165     class VideoCandidates:
166         """One video's persisted detection, ready for offline re-scoring.
167
168         ``intervals`` are the candidate windows; ``features`` is column-major
169         (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
170         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171         with ``load_video_candidates``, or directly in tests.
172         """
173
174         name: str
175         intervals: List[Interval]
176         features: Dict[str, List[float]] = field(default_factory=dict)
177         gts: Optional[List[Interval]] = None
178
179
180     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182         ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183         col = vc.features.get(name)
184         n = len(vc.intervals)
185         if col is None:
186             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                             name, vc.name)
188             return [0.0] * n
189         if len(col) != n:
190             raise ExperimentError(
191                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192         return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196         cols = [_feature_column(vc, name) for name in feature_names]
197         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202         players)."""
203         n = len(vc.intervals)
204         mask = [True] * n
205         if variant.min_crossings > 0:
206             col = _feature_column(vc, "net_crossings")
207             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208         if variant.min_players > 0:
209             col = _feature_column(vc, "players_in_court")
210             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211         return mask
212

```

```

213 def predict(variant: Variant, vc: VideoCandidates,
214             model: Optional[LogisticRegression] = None,
215             threshold: Optional[float] = None,
216             calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217     """Variant prediction: gate by persisted features, optionally filter by a learned
218     model, then merge. Pure and deterministic.
219
220     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225     them.
226     """
227     mask = _gate_mask(variant, vc)
228     if variant.feature_names:
229         if model is None:
230             if variant.classifier_model is None:
231                 raise ExperimentError(
232                     f"variant {variant.name!r} is learned but has no model to predict "
233                     f"with (pass a fitted model or set classifier_model)")
234             model = LogisticRegression.load(variant.classifier_model)
235             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
236 variant.feature_names))]
237             if calibrator is not None:
238                 proba = [calibrator(p) for p in proba]
239             thr = variant.threshold if threshold is None else threshold
240             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
241             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
242             return merge_overlaps(kept, gap_tol=variant.merge_gap)
243
244 def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
245                        train_videos: Sequence[VideoCandidates], iou: float
246                        ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
247     """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
248     (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
249     (use the variant default). Honest: never looks at the held-out video.
250     """
251     calibrator: Optional[Callable[[float], float]] = None
252     if variant.calibrate:
253         raw: List[float] = []
254         y: List[int] = []
255         for vc in train_videos:
256             raw.extend(float(p) for p in
257 model.predict_proba(_feature_matrix(vc, variant.feature_names or
258 [])))
259             y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
260         if variant.calibrate == "platt":
261             calibrator = fit_platt(raw, y)
262         elif variant.calibrate == "isotonic":
263             calibrator = fit_isotonic(raw, y)
264         else:
265             raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
266 'platt'/'isotonic')")
267     threshold: Optional[float] = None
268     if variant.tune_threshold:
269         best_t, best_f1 = variant.threshold, -1.0
270         for k in range(1, 20):
271             # grid 0.05..0.95 on the train
272             fold's rally F1
273             t = round(0.05 * k, 2)
274             f1s = [score(predict(variant, vc, model=model, threshold=t,
275 calibrator=calibrator),
276 vc.gts or [], iou).f1 for vc in train_videos]

```

```

271         mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272         if mean_f1 > best_f1:
273             best_f1, best_t = mean_f1, t
274         threshold = best_t
275     return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (`classifier_model` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:

```

```

329         others = [v for j, v in enumerate(videos) if j != i]
330         if not others:
331             raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332         model: Optional[LogisticRegression] = _train(variant, others, iou)
333         if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334             assert model is not None # _train never returns
None
335             cal, thr = _calibrate_and_tune(variant, model, others, iou)
336         elif recipe_lovo: # honest=False ? one model over all
videos
337             model = all_model
338         else: # static variant or pinned model
339             model = pinned_model
340         preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341         assert vc.gts is not None # guarded at function top
342         sc = score(preds, vc.gts, iou)
343         per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344         predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                     for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382

```

```

383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                        eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
387         EXPERIMENT_HARNESS §6).
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
389         order),
390         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
391         when
392         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
393         """
394         a_f1 = [p["f1"] for p in eval_a["per_video"]]
395         b_f1 = [p["f1"] for p in eval_b["per_video"]]
396         return {
397             "variant_a_ci": _ci_block(eval_a),
398             "variant_b_ci": _ci_block(eval_b),
399             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
400             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
401                             "variant_b": _segmentation_block(videos, eval_b)},
402         }
403
404     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
405                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
406     Dict[str, Any]:
407         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
408         same
409         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
410
411         The deltas use the existing `metrics.score` (per video) +
412         `calibration.pct_improvement`;
413         learned variants are scored LOVO-honestly. The result is a self-contained record fit
414         for
415         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
416         it.
417         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
418         CIs, a
419         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
420         *alongside* the
421         existing bare-mean fields (additive: nothing existing changes). Set False for the
422         legacy
423         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
424         §13/C1+C4): a
425         bare LOVO mean at n?6 is not trustworthy on its own.
426         """
427         if len(videos) < 1:
428             raise ExperimentError("compare needs at least one labeled video")
429         eval_a = evaluate_variant(videos, variant_a, iou, honest)
430         eval_b = evaluate_variant(videos, variant_b, iou, honest)
431
432         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
433         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
434                     m in _METRICS}
435
436         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
437         per_video_delta = [
438             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
439             for p in eval_b["per_video"]]
440         ]
441
442         # One fingerprint over the whole label set ? detects "the deltas were measured
443         against
444         # different labels" the same way regression.gt_hash guards the no-regression

```

```

baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451     # ----- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                          agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465           the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
468         a
469         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470         ``metrics.match_intervals`` ? no new matching logic.
471
472         """
473         preds_a = predict(variant_a, video)
474         preds_b = predict(variant_b, video)
475         mr = match_intervals(preds_a, preds_b, agree_iou)
476
477         agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
478                  for m in mr.matches]
479         agree_iou = [m.iou for m in mr.matches]
480         a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
481         b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
482
483         def _dur(ivs: List[Interval]) -> float:
484             return sum(e - s for s, e in ivs)
485
486         return {
487             "video": video.name,
488             "agree_iou": agree_iou,
489             "variant_a": variant_a.name,
490             "variant_b": variant_b.name,
491             "num_a": len(preds_a),
492             "num_b": len(preds_b),
493             "num_agreed": len(agreed),
494             "mean_agree_iou": (sum(agree_iou) / len(agree_iou)) if agree_iou else 0.0,
495             "duration_a": _dur(preds_a),
496             "duration_b": _dur(preds_b),
497             "agreed": agreed,
498             "a_only": a_only,

```

```

497         "b_only": b_only,
498     }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                     timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                     "spec_hash": result["variant_a"]["spec_hash"],
518                     "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                     "spec_hash": result["variant_b"]["spec_hash"],
521                     "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)
535     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536     with open(path, "w", encoding="utf-8") as f:
537         json.dump(entries, f, indent=2)
538     logger.info("recorded experiment %s vs %s (?f1=%3f) -> %s",
539               row["variant_a"]["name"], row["variant_b"]["name"],
540               (result.get("delta") or {}).get("f1", 0.0), path)
541     return row
542
543 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                         gts: Optional[List[Interval]] = None,
545                         feature_names: Sequence[str] = ()) -> VideoCandidates:
546     """Bridge persisted detection ? `VideoCandidates` via
547     `Database.query_candidate_segments`.
548     ``feature_names`` selects which persisted features to lift out of each row's
549     ``stats``
550     JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551     touching
552     function; the comparison itself is pure and offline. Exercised owner-side on a real
553     DB,
554     but works against any `Database` (including an in-memory one in tests).
555     """
556     rows = db.query_candidate_segments(video_id, detector=detector)
557     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558     rows]
559     features: Dict[str, List[float]] = {}

```

```

556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

14. assistant (2026-06-14T06:32:45.443Z)

I have a complete picture of the decision path. Now let me verify the "retained" criterion ?
check git history / docs to confirm nothing deleted the audio code after the n=6 ?F1 was non-
significant, and check the test file confirms the feature math is exercised.

15. user (2026-06-14T06:32:46.393Z)

```

1      """Tests for the P4 audio cue ? pure audio features + the incremental-lift driver (no
GPU/ffmpeg)."""
2      import json
3      import os
4      from types import SimpleNamespace
5
6      import pytest
7
8      from backend.eval import fusion_audio
9      from backend.pipeline.detectors import audio_features as af
10
11
12      def H(t, strength=1.0):
13          return SimpleNamespace(t=t, strength=strength)
14
15
16      def test_compute_audio_features_empty():
17          f = af.compute_audio_features([], 0.0, 10.0)
18          assert set(f) == set(af.AUDIO_FEATURE_NAMES)
19          assert all(v == 0.0 for v in f.values())
20
21
22      def test_compute_audio_features_counts_rate_strength():
23          hits = [H(1.0, 2.0), H(2.0, 4.0), H(5.0, 6.0), H(50.0, 1.0)] # 3 in [0,10), 1
outside
24          f = af.compute_audio_features(hits, 0.0, 10.0)
25          assert f["audio_hit_count"] == 3.0
26          assert f["audio_hit_rate"] == pytest.approx(0.3) # 3 hits / 10 s
27          assert f["audio_hit_strength_mean"] == pytest.approx((2 + 4 + 6) / 3)
28
29
30      def test_compute_audio_features_half_open_boundary():
31          hits = [H(10.0), H(20.0)] # window [10,20): 10
in, 20 out
32          f = af.compute_audio_features(hits, 10.0, 20.0)
33          assert f["audio_hit_count"] == 1.0
34
35
36      # ----- incremental-lift driver
(synthetic)
37
38      def _write_video(d, name):
39          """shuttle + person constant (no signal); audio_hit_count separates rallies (8) from
junk (0)."""
40          cands = []
41          for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,

```



```

51.0), 0)]:
52     shuttle = {fn: 1.0 for fn in fusion_audio.SHUTTLE_FEATURE_NAMES}
53     person = {fn: 1.0 for fn in fusion_audio.PERSON_FEATURE_NAMES}
54     audio = {fn: 0.0 for fn in af.AUDIO_FEATURE_NAMES}
55     audio["audio_hit_count"] = 8.0 if pos else 0.0
56     cand.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
57                 "audio": audio, "label": pos})
58     data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
59            "candidates": cand, "gts": [[0.0, 10.0], [20.0, 30.0]]}
60     with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
61         json.dump(data, f)
62
63     def test_audio_increment_compare(tmp_path):
64         d = str(tmp_path)
65         for k in range(6):
66             _write_video(d, f"v{k}")
67         videos = fusion_audio.load_videos_with_audio(d)
68         assert len(videos) == 6
69         res = fusion_audio.compare_increment(videos, iou=0.5)
70         assert res["variant_b"]["honest_lovo"] is True
71         assert res["delta"]["f1"] > 0.0 # audio adds separating signal on top of
72         constant shuttle+person
73         assert "audio" in fusion_audio.format_result(res, 0.5).lower()
74
75     def test_load_videos_requires_audio(tmp_path):
76         data = {"name": "x", "fps": 30.0, "frame_width": 1920.0,
77                "candidates": [{"start": 0.0, "end": 1.0,
78                               "shuttle": {fn: 0.0 for fn in
79 fusion_audio.SHUTTLE_FEATURE_NAMES},
80                "person": {fn: 0.0 for fn in
81 fusion_audio.PERSON_FEATURE_NAMES},
82                "label": 0}],
83                "gts": [[0.0, 1.0]]}
84         with open(os.path.join(str(tmp_path), "x_golden_features.json"), "w",
85 encoding="utf-8") as f:
86             json.dump(data, f)
87         with pytest.raises(SystemExit):
88             fusion_audio.load_videos_with_audio(str(tmp_path))
89
90

```

16. user (2026-06-14T06:32:47.995Z)

d7b1f51 feat(eval): honest ?F1 ? bootstrap CI + paired sign test on the fusion lift (#127)
dc19bf2 feat(eval): fusion P4 ? audio cue + incremental-lift measurement (default-OFF) (#122)
df9cf80 ci: add mypy lane (lenient green baseline + explicit quarantine list) (#93)
8615534 refactor: adopt approved layered directory structure

17. assistant (2026-06-14T06:32:56.613Z)

Let me confirm the "retained despite non-significant n=6" claim by checking the
QUALITY_ITERATIONS / audio-e1-findings docs and that there is no deletion commit removing audio.
Let me also verify there's no audio in the served run path and check the memory note on audio.

18. user (2026-06-14T06:32:58.826Z)

---DELETIONS-ABOVE (empty=None)---
9eb06f4 docs(quality): log fusion P3+P4 measured ?F1 on the n=6 golden corpus
fa12829 chore: gitignore scratch/ to keep git status clean (#136)
55df82f feat(obs): run-telemetry black box ? fps + GPU temp/clocks/power + CPU/RAM (rotating
~10KB JSONL) (#134)
a9432a9 feat(db): self-scheduling job substrate ? migration v3 + claim_due_job (P2) (#135)

ec2002d feat(infra): ExecutionPolicy P1 ? bound the Gemini hang (op_timeout + bounded poll)
(#133)

19. user (2026-06-14T06:33:00.123Z)

```
43:### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising  
(not promotable) *(2026-06-14)*  
52:| + person + audio (P4 incr.) | 0.366 | **+0.079** (+27.7%) | [?0.054, +0.203] | 4W/2L  
(p=0.688) | **No** |  
56:per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising lead  
? +0.079 / +27.7%, wins 4/6  
57:(largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above**  
shuttle-only (0.307) ? but n=6 can't  
58:confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and  
measure audio on shuttle-only  
107: + `brier_score`/`expected_calibration_error`, so a CNN softmax, an audio onset, and an  
integer net-crossing  
198: for the same reason (foreground spatial gate **?0.04**; audio energy **AUC ~0.6**; court-  
region
```

20. assistant (2026-06-14T06:33:02.996Z)

This confirms everything. Let me read the exact QUALITY_ITERATIONS lines around the n=6 audio result to nail the "retained, re-measurable" evidence precisely.

21. user (2026-06-14T06:33:03.872Z)

```
43      ### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio =  
promising (not promotable) *(2026-06-14)*  
44      The GPU person-feature extraction finished all 6 golden videos (`fusion_golden.py` ?  
`output/*_golden_features.json`, the  
45      replayable substrate); the CPU LOVO drivers then measured the lift with **honest stats  
(C1 ? bootstrap 95% CI + paired sign  
46      test; never a bare mean at n=6)**:  
47  
48      | Variant (honest LOVO, IoU ? 0.5) | F1 | ?F1 vs prior | 95% CI | sign test | CI clears  
0 |  
49      |---|---|---|---|---|  
50      | shuttle-only | 0.307 | ? | ? | ? | ? |  
51      | + person (P3) | 0.286 | **?0.021** (?6.7%) | [?0.165, +0.162] | 2W/4L (p=0.688) |  
**No** |  
52      | + person + audio (P4 incr.) | 0.366 | **+0.079** (+27.7%) | [?0.054, +0.203] | 4W/2L  
(p=0.688) | **No** |  
53  
54      **Read honestly:** neither cue clears the promotion bar (both CIs span 0, both sign  
tests n.s.) at n=6 ? **both stay  
55      default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ?  
`players_in_court` counts spectators);  
56      per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising  
lead ? +0.079 / +27.7%, wins 4/6  
57      (largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above**  
shuttle-only (0.307) ? but n=6 can't  
58      confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and  
measure audio on shuttle-only  
59      *directly* (without the person handicap). Recorded in  
`eval_baselines/experiment_leaderboard.json`. **Negative/uncertain  
60      results are kept on purpose ? this is the honest ablation backbone for the paper aim.**  
61  
62      ### Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125  
corroboration *(2026-06-14)*  
63      **Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness).** The  
measured win  
64      (?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg  
1.68?1.01 ? the
```

22. assistant (2026-06-14T06:33:09.855Z)

I now have complete, concrete evidence for all four criteria. Let me also do a final negative check to confirm the audio cue does NOT appear anywhere in the served production path (main.py, the segmenter pipeline, providers) ? i.e., the only importers of the audio modules are eval/test code.

23. user (2026-06-14T06:33:11.819Z)

---NON-EVAL IMPORTS ABOVE (empty=audio is eval-only)---

24. assistant (2026-06-14T06:33:27.600Z)

Confirmed: the audio modules have ZERO importers in the served/production path. Every consumer is in `backend/eval/` or `tests/`. I now have all evidence needed for the four criteria. Let me compile the structured finding.

Summary of the audit:

****present**** ? YES. `audio\_features.py` (AUDIO_FEATURE_NAMES + compute_audio_features) and `hit\_detector.py` (full onset-detection chain ending in `detect\_hits\_in\_video`) both exist and are unit-tested.

****default_off**** ? YES, strongly. The served path is selected by `default\_segmenter` (`yolo\_hybrid` in model, `gemini` in config.json) ? never the fusion segmenter, and even the opt-in `fusion\_hybrid` segmenter never touches audio. The audio cue's only consumers are `backend/eval/fusion\_audio.py` and `backend/eval/audio/e1\_probe.py`. The P4 variant adds audio only as extra `feature\_names` columns (default-OFF; `git grep` confirms zero non-eval importers).

****no_special_casing**** ? YES. Audio enters strictly as three feature columns the learned `LogisticRegression` weights via `experiment.predict` ? `\_feature\_matrix`. There is no `if audio\_hit\_count > X` branch anywhere in the decision path. `compute\_audio\_features` returns a numeric dict; the gate masks (`\_gate\_mask`) only reference `net\_crossings`/`players\_in\_court`, not audio.

****retained**** ? YES. No deletion commit exists (diff-filter=D empty). After the n=6 ?F1 (+0.079, CI [?0.054,+0.203], sign test 4W/2L p=0.688 ? non-significant), QUALITY_ITERATIONS.md explicitly states "both stay default-OFF" and "Negative/uncertain results are kept on purpose." Producer + feature math both live; re-measurable via `fusion\_audio.py --augment --compare` when N grows.

Concerns: (1) the producer/feature math is fully exercised only by synthetic tests ? `detect\_hits\_in\_video` (real ffmpeg+audio) has no offline test (the container lacks ffmpeg/numpy for it), so the real-audio path is owner-side-verified only; (2) there is no `cue\_flags["audio"]` and `fusion\_hybrid.py` has no audio enrichment seam, so even if measured-good, audio is NOT yet wired to the served segmenter (eval-only) ? promotion would need new wiring, unlike Gate A which already has `min\_crossings`.

25. user (2026-06-14T06:33:51.898Z)

Structured output provided successfully

26. assistant (2026-06-14T06:33:55.188Z)

Audit complete. All four criteria for the AUDIO cue are satisfied (present, default_off, no_special_casing, retained), with two substantive concerns flagged: the real-audio producer path is owner-side-verified only (no offline test), and audio is not yet wired to any served segmenter (eval-harness-only, unlike Gate A/person which have served seams).